Python Data Engineering & EDA Cheat Sheet

Focus: Python libraries and methods commonly used for data exploration, cleaning, transformation, validation, visualization, and Data Engineering workflows.

1. Core Python Libraries

Library	Primary use
pandas	DataFrames, EDA, cleaning, transformation
numpy	Numerical computing and arrays
matplotlib	Core visualization
seaborn	Statistical visualization
scipy	Statistics and scientific computing
statsmodels	Statistical modelling
os / pathlib / glob	Files and directories
json / csv	Structured file formats
datetime	Dates and timestamps
re	Regular expressions / text cleaning
logging	Pipeline logging

2. NumPy — Numerical Foundation

```
import numpy as np

np.array([1,2,3])
np.zeros(5)
np.ones(5)
np.arange(0,10,2)
np.linspace(0,1,5)
np.random.randint(1,100,10)

arr.shape
arr.ndim
arr.size
arr.dtype

arr.reshape(2,5)
arr.flatten()
arr.ravel()

np.sum(arr)
np.mean(arr)
np.median(arr)
np.std(arr)
np.var(arr)
np.min(arr)
np.max(arr)
np.percentile(arr,75)

arr[arr > 50]
np.where(arr > 50, "High", "Low")
np.unique(arr)
np.sort(arr)
np.argsort(arr)
np.isnan(arr)
np.isinf(arr)
```

3. Pandas — Core EDA Library

```
import pandas as pd

# Read
pd.read_csv("data.csv")
pd.read_excel("data.xlsx")
pd.read_json("data.json")
pd.read_parquet("data.parquet")
pd.read_sql(query, connection)

# Write
df.to_csv("output.csv", index=False)
df.to_excel("output.xlsx", index=False)
df.to_json("output.json")
df.to_parquet("output.parquet", index=False)
```

4. Dataset Inspection

```
df.head()
df.tail()
df.sample(10)
df.shape
df.columns
df.index
df.dtypes
df.info()
df.describe()
df.describe(include="all")

df.nunique()
df["column"].nunique()
df["column"].unique()
df["column"].value_counts()
df["column"].value_counts(normalize=True)
```

5. Selecting & Filtering

```
df["age"]
df[["name","age","salary"]]

df.iloc[0]
df.iloc[0:10]
df.loc[0:10]

df[df["age"] > 30]
df[(df["age"] > 30) & (df["salary"] > 50000)]
df[(df["city"] == "Chennai") | (df["city"] == "Hyderabad")]

df.query("age > 30 and salary > 50000")
```

6. Missing Data

```
df.isnull()
df.isna()
df.isnull().sum()
df.isnull().mean() * 100

df.dropna()
df.dropna(axis=1)
df.dropna(subset=["age"])

df["age"].fillna(0)
df["age"].fillna(df["age"].mean())
df["age"].fillna(df["age"].median())
df["city"].fillna("Unknown")

df.ffill()
df.bfill()
```

7. Duplicates & Data Types

```
df.duplicated()
df.duplicated().sum()
df.drop_duplicates()
df.drop_duplicates(subset=["customer_id"])

df.dtypes
df["age"] = df["age"].astype(int)
df["salary"] = pd.to_numeric(df["salary"])
df["date"] = pd.to_datetime(df["date"])
df["category"] = df["category"].astype("category")

df.select_dtypes(include="number")
df.select_dtypes(include="object")
```

8. String & Regex Operations

```
df["name"].str.lower()
df["name"].str.upper()
df["name"].str.title()
df["name"].str.strip()
df["name"].str.len()
df["name"].str.contains("john")
df["name"].str.startswith("A")
df["name"].str.endswith("n")
df["name"].str.split("@")
df["name"].str.replace("old","new")

df["email"].str.extract(r"(@.*)$")
df["phone"].str.replace(r"\D","",regex=True)
```

9. DateTime

```
df["date"] = pd.to_datetime(df["date"])

df["date"].dt.year
df["date"].dt.month
df["date"].dt.day
df["date"].dt.day_name()
df["date"].dt.weekday
df["date"].dt.quarter

df[df["date"] >= "2026-01-01"]

df["delivery_days"] = (
df["delivery_date"] - df["order_date"]
).dt.days
```

10. Sorting, Aggregation & GroupBy

```
df.sort_values("salary")
df.sort_values("salary", ascending=False)
df.sort_values(["department","salary"], ascending=[True,False])

df["salary"].sum()
df["salary"].mean()
df["salary"].median()
df["salary"].min()
df["salary"].max()
df["salary"].std()

df.groupby("department")["salary"].mean()

df.groupby("department")["salary"].agg(
["count","mean","min","max","sum"]
)

df.groupby("department").agg(
avg_salary=("salary","mean"),
max_salary=("salary","max"),
employee_count=("employee_id","count")
)
```

11. Crosstab & Pivot Tables

```
pd.crosstab(df["gender"], df["department"])

pd.pivot_table(
df,
values="salary",
index="department",
columns="gender",
aggfunc="mean"
)
```

12. Combining Data

```
pd.concat([df1, df2])
pd.concat([df1, df2], axis=1)

pd.merge(
customers,
orders,
on="customer_id",
how="inner"
)

# Common joins:
# inner, left, right, outer, cross

df1.join(df2)
```

13. Transformations

```
df["salary"].apply(lambda x: x * 1.10)

df["gender"].map({
"M": "Male",
"F": "Female"
})

df["city"].replace({
"Hyd": "Hyderabad",
"Chennai ": "Chennai"
})

df = df.assign(
annual_salary=df["monthly_salary"] * 12
)
```

14. Ranking & Window-like Operations

```
df["salary_rank"] = df["salary"].rank(ascending=False)

df.nlargest(10, "salary")
df.nsmallest(10, "salary")

df["previous_sales"] = df["sales"].shift(1)
df["sales_change"] = df["sales"].diff()
df["running_total"] = df["sales"].cumsum()
df["running_max"] = df["sales"].cummax()

df["rolling_avg"] = df["sales"].rolling(7).mean()

df["dept_avg"] = (
df.groupby("department")["salary"]
.transform("mean")
)
```

15. Statistical EDA

```
df["salary"].mean()
df["salary"].median()
df["salary"].mode()
df["salary"].std()
df["salary"].var()
df["salary"].min()
df["salary"].max()

df["salary"].quantile(0.25)
df["salary"].quantile(0.50)
df["salary"].quantile(0.75)

df["salary"].skew()
df["salary"].kurtosis()

df.corr(numeric_only=True)
df["age"].corr(df["salary"])
df.cov(numeric_only=True)
```

16. Outlier Detection — IQR

```
Q1 = df["salary"].quantile(0.25)
Q3 = df["salary"].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[
(df["salary"] < lower) |
(df["salary"] > upper)
]
```

17. Matplotlib

```
import matplotlib.pyplot as plt

plt.plot(df["date"], df["sales"])
plt.bar(df["department"], df["salary"])
plt.hist(df["salary"], bins=20)
plt.scatter(df["age"], df["salary"])
plt.boxplot(df["salary"])
plt.show()
```

18. Seaborn

```
import seaborn as sns

sns.histplot(data=df, x="salary", kde=True)

sns.boxplot(
data=df,
x="department",
y="salary"
)

sns.countplot(data=df, x="department")

sns.scatterplot(
data=df,
x="age",
y="salary",
hue="department"
)

sns.heatmap(
df.corr(numeric_only=True),
annot=True
)
```

19. Files & Directories

```
import os
os.getcwd()
os.listdir()
os.path.exists("data.csv")
os.path.join("data","sales.csv")

from pathlib import Path
path = Path("data")
list(path.glob("*.csv"))

import glob
files = glob.glob("data/*.csv")

dfs = [pd.read_csv(file) for file in files]
final_df = pd.concat(dfs)
```

20. JSON

```
import json

with open("data.json") as f:
data = json.load(f)

with open("output.json", "w") as f:
json.dump(data, f)

pd.read_json("data.json")
```

21. SQL + Python

```
import pandas as pd
from sqlalchemy import create_engine

engine = create_engine(
"postgresql://user:password@host:5432/database"
)

df = pd.read_sql(
"SELECT * FROM customers",
engine
)

df.to_sql(
"customers",
engine,
if_exists="append",
index=False
)

# Drivers:
# PostgreSQL → psycopg
# MySQL → mysql-connector-python
# SQL Server → pyodbc
# SQLite → sqlite3
```

22. Parquet & PyArrow

```
df.to_parquet("sales.parquet", index=False)
df = pd.read_parquet("sales.parquet")

import pyarrow as pa
import pyarrow.parquet as pq
```

Why Parquet? Column-oriented storage, compression, faster analytical reads, and common usage in Data Lakes and distributed data platforms.

23. Polars

```
import polars as pl

df = pl.read_csv("sales.csv")
df = pl.read_parquet("sales.parquet")

df.filter(pl.col("sales") > 1000)

df.group_by("department").agg(
pl.col("salary").mean()
)

# Pandas → primary EDA skill
# Polars → high-performance DataFrames
# PySpark → distributed processing
```

24. PySpark — Data Engineering Essential

```
from pyspark.sql import SparkSession
```

```
spark = (
SparkSession.builder
.appName("DataEngineering")
.getOrCreate()
)

df = spark.read.csv(
"sales.csv",
header=True,
inferSchema=True
)

df.show()
df.printSchema()
df.count()

df.select("customer_id", "sales")
df.filter(df["sales"] > 1000)
df.groupBy("department").avg("salary")
```

25. Data Validation

Production checks: nulls, duplicates, data types, ranges, uniqueness, referential integrity, schema validation, and business rules.

```
assert df["customer_id"].notna().all()
assert df["customer_id"].is_unique
assert (df["age"] >= 0).all()

# Useful libraries:
# Pandera
# Great Expectations
# Pydantic
```

26. Logging

```
import logging

logging.basicConfig(level=logging.INFO)

logging.info("Pipeline started")
logging.warning("Missing values detected")
logging.error("Pipeline failed")
```

27. EDA Workflow — Memorize

LOAD → UNDERSTAND → CLEAN → VALIDATE → EXPLORE → TRANSFORM → VISUALIZE → IDENTIFY INSIGHTS → EXPORT

```
df = pd.read_csv("data.csv")

df.shape
df.info()
df.describe()

df.isna().sum()
df.duplicated().sum()
df.nunique()

df["sales"].describe()

df.groupby("category")["sales"].sum()

df.corr(numeric_only=True)

sns.histplot(data=df, x="sales")

df.to_parquet("clean_data.parquet")
```

28. Recommended Learning Priority for Data Engineering

Tier	Learn / Master
Tier 1 — Master	Python, Pandas, NumPy, SQL, pathlib, JSON, datetime
Tier 2 — Strong working knowledge	Matplotlib, Seaborn, SQLAlchemy, PyArrow, PySpark, boto3, logging, re
Tier 3 — Learn as needed	Polars, SciPy, Statsmodels, Pandera, Great Expectations, Pydantic

29. Data Engineering Python Stack

Python → Pandas → NumPy → SQL → PyArrow/Parquet → PySpark → boto3 → SQLAlchemy → Data Validation → Logging

Typical flow: Python → EDA/cleaning → Parquet/S3 → AWS Glue/PySpark → RDS/warehouse → dbt → Tableau.